

## BM 321 Yazılım Mühendisliği - Proje Öneri ve Sistem Vizyonu Dokümanı

**Proje Adı:** SimullInvest: Yapay Zeka Destekli Sanal Borsa ve Portföy Simülasyonu

**Tarih:** 10.10.2026

### **Takım:**

Eren Özdemir 240206004

Kerem Türkyılmaz 240206016

Mine Yamaner 240206018

Özge Karaca 240206024

Fatma Saruhan 240206027

**Sürüm:** 1.1

**GitHub Repo:** [github.com/BM321-2026-Guz/SimullInvest](https://github.com/BM321-2026-Guz/SimullInvest)

### **1. Problem Tanımı ve Proje Amacı**

Günümüzde finansal okuryazarlığı düşük olan yeni yatırımcılar, piyasaları öğrenmek için genellikle gerçek birikimlerini riske atmak zorunda kalmaktadır. Piyasada bulunan mevcut borsa uygulamaları ise yeni başlayanlar için karmaşık grafikler ve terimlerle doludur.

**Projenin Amacı:** Kullanıcıların kendilerine başlangıçta tanımlanan sanal bakiye ile Borsa (Hisse Senetleri), Kripto Paralar ve Emtia (Altın vb.) piyasalarında risksiz ve güncel piyasa fiyatlarıyla işlem yapabilmelerini sağlamaktır. Sistem, "API-First" mimarisiyle küresel fiyatları dış veri sağlayıcılardan belirli aralıklarla (önbellekli) çekecek ve Yapay Zeka destekli haber duygu analizi (Sentiment Analysis) ile kullanıcılara finansal okuryazarlık kazandıracaktır.

**Değer Önerisi:** SimullInvest, yatırıma yeni başlayanlara gerçek para riski olmadan, sade bir arayüz ve yapay zeka destekli haber yorumlarıyla piyasayı öğrenme imkânı sunar.

**Hedef Kitle:** Finansal okuryazarlığı düşük, yatırıma yeni başlamak isteyen üniversite öğrencileri ve genç yetişkinler.

### **Başarı Ölçütleri:**

- Kullanıcı bakiyesi hiçbir koşulda negatif olamaz; yetersiz bakiyeyle yapılan alım reddedilir.
- Kullanıcı kayıttan sonra 3 dakika içinde ilk alım işlemini gerçekleştirebilir.
- Varlık fiyatları en fazla 1 dakika gecikmeyle sunulur.

### **2. Sistem Vizyonu ve Sınırları**

Geliştirilecek platform, "Kapsam Kayması (Scope Creep)" riskini önlemek amacıyla katı sistem sınırlarına sahiptir.

- **Sisteme Dahil Olanlar:** Sanal cüzdan yönetimi, dış veri sağlayıcılardan (örn. Yahoo Finance / yfinance, CoinGecko) REST API ile güncel fiyat çekilmesi, alım-satım emirlerinin işlenmesi, yapay zeka entegreli haber akışı ve portföy kâr/zarar hesaplamaları.
- **Sistem Sınırları Dışında Kalanlar:** Gerçek banka hesapları veya kredi kartı entegrasyonu, otomatik limit/stop-loss emirleri ve saniyelik canlı fiyat akışları (WebSocket) sistemin dışındadır. Fiyatlar belirli aralıklarla (örn. 60 saniye) güncellenir ve önbelleğe alınır. Uygulama sadece bir simülasyondur.
- **Varsayımlar:** Tüm parasal değerler tek bir TL cüzdan üzerinden tutulur. Dış veri sağlayıcı kesintisinde önbellek ve yedek (mock) veri kullanılır.

### 3. Takım Yapılanması, Roller ve RACI Matrisi

Proje ekibi 5 kişiden oluşmakta olup, takım içi uzmanlaşmayı sağlamak amacıyla aşağıdaki zorunlu görev dağılımı yapılmıştır:

- **Product Owner:** Fatma Saruhan — Kullanıcı hikayelerinin ve kabul kriterlerinin yazılması, GitHub Projects üzerinde Backlog'un önceliklendirilmesi.
- **Scrum Master:** Kerem Türkyılmaz — Çevik süreçlerin yürütülmesi, günlük toplantıların organizasyonu ve engelleyicilerin çözülmesi.
- **Lead Software Architect:** Özge Karaca — Sistem mimarisinin (C4 Modeli) tasarlanması, kod standartlarının belirlenmesi ve PR incelemeleri.
- **QA & Test Automation Eng.:** Mine Yamaner — Test stratejisinin kurulması, %50 üzeri kapsamla (Coverage) Birim (Unit) ve E2E testlerinin yazılması.
- **DevOps & CI/CD Eng.:** Eren Özdemir — CI/CD pipeline kurulumu, Dockerfile/docker-compose yapılandırması ve otomatik derleme süreçleri

**Not:** Takım 5 kişidir ve ayrı bir Security (DevSecOps) rolü bulunmamaktadır. OWASP Top 10 analizi Scrum Master tarafından, SAST (CodeQL) taraması DevOps mühendisi tarafından yürütülür. Frontend geliştirmeden Scrum Master sorumludur. Ayrıntılı matris için docs/raci.md dosyasına bakınız.

**RACI (Sorumlu, Onaylayan, Danışılan, Bilgilendirilen) Görev Dağılım Matrisi:**

| Proje Görevleri / İş Paketleri    | Fatma (PO) | Kerem (SM) | Özge (Arch) | Mine (QA) | Eren (DevOps) |
|-----------------------------------|------------|------------|-------------|-----------|---------------|
| Gereksinimler & User Stories      | R/A        | C          | I           | I         | I             |
| Agile Süreçleri & Sprint Planlama | C          | R/A        | I           | I         | I             |
| Sistem Mimarisi & API Tasarımı    | I          | C          | R/A         | C         | C             |
| Birim ve Uçtan Uca (E2E) Testler  | I          | I          | C           | R/A       | C             |
| CI/CD Pipeline & Docker Kurulumu  | I          | I          | C           | C         | R/A           |
| Frontend Geliştirme               | C          | R/A        | C           | I         | I             |
| OWASP Top 10 Güvenlik Analizi     | I          | R/A        | C           | I         | C             |
| SAST Taraması (CodeQL)            | I          | I          | C           | C         | R/A           |

**Not:** R: Sorumlu (iş yapan), A: Onaylayan (hesap veren), C: Danışılan, I: Bilgilendirilen. Güncel ve ayrıntılı matris docs/raci.md dosyasında tutulmaktadır.

#### 4. Taslak OpenAPI / Swagger API Sözleşmesi

Sistem, "API-First" tasarım felsefesine uygun olarak geliştirilecektir. Backend ve Frontend arasındaki iletişimi standartlaştırmak için taslak API sözleşmemiz hazırlanmıştır.

Şartname gereği sözleşmenin tam ve çalışır hali, GitHub depomuzda docs/openapi.yaml dizini altında sunulmuştur. Sistemin temel altyapısını ve veri dönüş tiplerini gösteren sözleşme taslağımızın çekirdek kısmı aşağıda verilmiştir:

**openapi: 3.0.3**

**info:**

**title: SimullInvest API**

**version: 1.1.0**

**paths:**

**/api/portfolio:**

**get:**

**summary: Kullanıcının güncel portföy durumunu ve sanal bakiyesini getirir.**

**responses:**

**'200':**

**description: Başarılı işlem.**

**content:**

**application/json:**

**schema:**

**type: object**

**properties:**

**balance:**

**type: number**

**example: 100000.00**

**holdings:**

**type: array**

**items:**

**\$ref: '#/components/schemas/Holding'**

**components:**

**schemas:**

**Holding:**

**type: object**

**properties:**

**symbol:**

**type: string**

**quantity:**

**type: number**

**averageCost:**

**type: number**

**Not:** Tüm parasal değerler TL cinsindendir. Portföydeki her varlık; sembol, adet ve ortalama maliyet bilgisiyle bir nesne olarak temsil edilir (tam sözleşme: docs/openapi.yaml).

#### Uç Nokta Özeti (docs/openapi.yaml):

| Yöntem | Yol                  | Açıklama                                           | Kimlik Doğrulama |
|--------|----------------------|----------------------------------------------------|------------------|
| POST   | /api/auth/register   | Yeni kullanıcı kaydı; 100.000 TL bakiye tanımlanır | Yok              |
| POST   | /api/auth/login      | Giriş yapar, JWT döner                             | Yok              |
| GET    | /api/portfolio       | Portföy ve bakiye durumu                           | JWT              |
| GET    | /api/assets          | Varlık fiyat listesi (tür filtresi ile)            | Yok              |
| GET    | /api/assets/{symbol} | Tek varlığın güncel fiyatı                         | Yok              |
| POST   | /api/trades          | Alım veya satım emri verir                         | JWT              |
| GET    | /api/transactions    | İşlem geçmişi                                      | JWT              |
| GET    | /api/news            | Haberler ve YZ duygu etiketi (P1)                  | Yok              |

## 5. GitHub Organization, Repo Kurulumu ve Git-Flow

Proje geliştirme süreçleri GitHub üzerinde organize edilmiş olup, aşağıdaki yapılandırmalar tamamlanmıştır:

- Repo Dosyaları:** Reponun kök dizininde projenin kurulum ve çalıştırma talimatlarını içeren README.md dosyası ile gereksiz dosyaların (örn. .env, \_\_pycache\_\_/, venv/, \*.db) repoya yüklenmesini engelleyen .gitignore dosyası oluşturulmuştur.
- Git-Flow Kuralları:**

- o **main:** Sadece çalışan, test edilmiş ve canlıya (Docker) çıkmaya hazır kodları barındırır. Doğrudan commit (push) yapılması yasaktır.
- o **develop:** Geliştirme sürecinin ana dalıdır. Tüm yeni özellikler burada birleştirilir.
- o **feature/X:** Yeni bir kullanıcı hikayesi geliştirilirken develop'tan açılan dallardır (Örn: feature/login-ekrani). İşlem bitince develop dalına Pull Request (PR) açılarak en az 1 inceleycinin (Tech Lead, müsait değilse Scrum Master) onayına sunulur.
- o **Koruma:** main ve develop dalları GitHub Ruleset ile korunmaktadır (Pull Request zorunlu, en az 1 onay, force push ve silme engelli).

## 6. Başlangıç Product Backlog'u ve İş Hiyerarşisi

Proje yönetimi GitHub Projects (Kanban) üzerinden araç bazlı olarak kurulmuştur. Görevler Epics → Stories → Tasks hiyerarşisine uygun şekilde etiketlenerek (Labels) önceliklendirilmiştir:

### Epic 1: Çekirdek Sistem (MVP - Zorunlu) (Etiket: P0-MustHave)

- **Story:** Kullanıcı kaydı, girişi ve güvenli oturum yönetimi. **(Story Point: 5)**
  - o *Task:* JWT entegrasyonu.
- **Story:** Yeni kullanıcılara 100.000 TL sanal bakiye tanımlanması. **(Story Point: 2)**
  - o *Task:* Veritabanı (DB) Cüzdan tablosunun oluşturulması.
- **Story:** Dış API bağlantısı ile güncel varlık fiyatlarının çekilmesi. **(Story Point: 5)**
- **Story:** Temel al-sat işlemlerinin veritabanına işlenmesi ve portföy kâr/zarar paneli. **(Story Point: 8)**

### Epic 2: İleri Düzey Mekanikler ve Cila Özellikleri (Etiket: P1-QuickWin)

- **Story:** Yapay Zeka (YZ) Duygu Analizi entegrasyonu (OpenAI/Gemini API ile finans haberlerinin "Yükseliş / Düşüş" olarak etiketlenmesi). **(Story Point: 8)**
- **Story:** Varlık dağılımını gösteren görsel pasta grafikler (Chart.js vb.). **(Story Point: 3)**
- **Story:** İzleme Listesi (Watchlist) oluşturulması. **(Story Point: 3)**

### Epic 3: Faz 2 Geliştirmeleri (Etiket: P2-NiceToHave)

- **Story:** Kullanıcılar arası rekabeti teşvik eden Liderlik Tablosu (Leaderboard). **(Story Point: 5)**
- **Story:** Seçili varlık için son 30 günlük geçmiş fiyat çizgi grafiği. **(Story Point: 3)**

**Not:** Story Point değerleri Fibonacci ölçeğiyle (1, 2, 3, 5, 8) yapılan ilk tahminlerdir; sprint planlamalarında takımca revize edilecektir. Epic'ler GitHub'da epic etiketli issue olarak, story ve task'lar ise bu issue'lara bağlı issue'lar (task list) olarak izlenir. Öncelik etiketleri P0/P1/P2, tür etiketleri epic/story/task'tır.

## 7. Etik Beyanı ve Lisans

- Bu proje geliştirilirken yazılım mühendisliği etik kurallarına bağlı kalınacak, herhangi bir kullanıcı verisi veya finansal veri kötüye kullanılmayacaktır. Uygulama, tamamen açık kaynaklı **MIT Lisansı** altında geliştirilecek olup, projenin kodları açık kaynak topluluğuna katkı sağlamak amacıyla herkese açık yayınlanacaktır.
- Uygulama yatırım tavsiyesi değildir; yalnızca eğitim amaçlı bir simülasyondur ve arayüzde bu yönde açık bir uyarı gösterilecektir.
- Kullanıcı verileri korunur: şifreler hash'lenerek saklanır, gerçek finansal veya kişisel kimlik bilgisi toplanmaz.
- API anahtarları ve gizli bilgiler repoya yüklenmez (.env dosyası ve GitHub Secrets kullanılır).
- Yapay zeka tarafından üretilen haber etiketleri hatalı olabilir; bu sınırlılık kullanıcıya belirtilir.